实验二

从零实现 BPE 分词器与自注意力

实验环境

- Python 3.11
- PyTorch 2.0 或更高版本
- pytest 8.0 或更高版本
- tiktoken 仅用于 BPE 对比

手写字节级 BPE 分词器

补全项目中 lab/student/bpe.py 标记的核心代码通过测试，并回答以下问题：

使用项目提供的测试程序进行验证：

python -m pytest tests/bpe -v

问题一：将 BPE 分词器的词表大小分别设置为 512、1024 和 2048，在同一测试语料上进行编码，记录 token 数量并计算压缩率（压缩率=原始字节数/编码token数）。请给出实验运行结果截图，截图中至少包含词表大小、原始文本字节数、Token 数量和压缩率，并将实验结果整理成表格。分析词表大小对分词粒度和压缩效果的影响，并讨论“词表是否越大越好”，说明原因。

问题二：分别使用自行实现的 BPE 分词器和 tiktoken 对给定的中文、英文以及中英混合文本进行编码和解码，记录 Token 数量，并验证解码结果是否与原文本完全一致。给出实验运行结果截图，并分析两种分词器 Token 数量存在差异的原因。

问题三：（思考题）对比自行实现的 BPE 与 tiktoken 的分词结果，分析训练语料、词表规模以及分词规则对 Token 划分结果的影响。能否仅根据 Token 数量判断一种分词器优于另一种？为什么？

多头注意力与掩码

补全项目中 lab/student/attention.py 标记的核心代码，完成缩放点积注意力、因果掩码和多头自注意力的实现。不得直接调用 PyTorch 标准注意力模块替代核心计算。
使用项目提供的测试程序进行验证：

```
python -m pytest tests/attention -v
```

实验报告需包含：
1. 实现思路：说明缩放点积注意力的计算流程、因果掩码的作用，以及多头注意力的拆分与拼接过程。
2. 关键代码截图：展示补全的核心代码，并配以简要说明。
3. 运行结果截图：展示注意力输出形状、注意力权重及未来位置权重为 0 的结果。
4. 测试通过截图：清晰展示测试命令、各项测试名称和全部通过的汇总结果。

实验要求与成果提交

提交物	必须包含的内容	验收标准
源代码	bpe.py attention.py	禁止用标准模块替代核心手写计算
实验报告	完成实验报告问题	结论与数据一致



截止时间：2026-10-08 23:59

提交地址：https://send2me.cn/maMQqRjb/Tmqjvc75ydypjA